Digital Images & the Pipeline

Week 1 — Session 1: Sampling, Quantization & Color

Practical Image Processing

Lecture 1.1 ▪ 60–70 min

Where this course is going

By the end you will build DocScan: a photo of a document → a clean, deskewed, machine-readable scan, then read it with a small neural net.

- Tools: **Python + OpenCV + NumPy**.
- Weeks 1–11: classical image processing.
- Weeks 12–14: an introduction to deep learning for vision.

The habit we build

Understand the pixels and the math *first*; call the library second; reach for a neural net only when classical methods run out.

The image-processing pipeline



Key idea

Every task in this course is some path through this pipeline. DocScan walks all of it.

Two sessions this week

Today — Session 1

What an image *is*. Sampling, quantization, resolution, bit depth, and color. The vocabulary and the numbers behind every pixel.

Next — Session 2

Images *in code*. NumPy arrays, formats, and the physical imaging model (noise, aliasing, gamma). Ends with the week's deliverable.

Same running examples throughout: a **water-meter** numerator and a license **plate** crop from our real dataset.

Roadmap

What is a digital image?

Color

In-class exercise

What is a digital image?

An image is a function, sampled onto a grid

- A continuous image is a function $f(x, y)$ = brightness at position (x, y).
- A **digital** image samples it on a grid and rounds each value: **pixels**.
- A grayscale image is a 2D array; a color image is 3 such arrays (channels).

Math

$f(x, y)$, with $x = 0 \dots W-1$, $y = 0 \dots H-1$, and each value in $\{0, \dots, 255\}$ (8-bit).

From continuous scene to discrete array



- **Sampling** chops space into a grid: *where* we measure.
- **Quantization** rounds each brightness to a level: *how precisely* we record it.
- Two knobs, two kinds of loss — the whole of this section.

Pixels are just numbers

A tiny 5×5 patch of intensities:

10	20	120	200	240
30	60	130	180	230
50	90	150	190	220
70	110	160	200	250
90,130	170,2	10,255		

- 0 = black, 255 = white, values between = gray.
- "Processing" an image = doing arithmetic on these numbers.
- There is no magic: every operation is math on an array.

Reading the grid: indexing a pixel

Same 5×5 patch, now with indices. In image code we write `img[row, col]`:

- `img[0,0]` = 10 (top-left).
- `img[2,3]` = 190 (row 2, column 3).
- `img[4,4]` = 255 (bottom-right, pure white).

10	20	120	200	240
30	60	130	180	230
50	90	150	190	220
70	110	160	200	250
90,130	170,210	255		

Note

Row first, then column. This is $[y, x]$, the *reverse* of math's (x, y) — more on that in Session 2.

A real image from our dataset



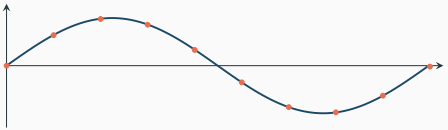
original (color) — a water-meter numerator



grayscale — one channel, same structure

This 48×196 crop is our running example. Reading these digits is exactly what the DocScan capstone builds toward. (Data: `E:/data/abfa_image/numerators`.)

Sampling: how finely we measure space



Sampling = choosing grid points (resolution). Too few samples and detail is lost — or worse, faked (aliasing, Session 2). More pixels = more spatial detail, more data.

Sampling, made concrete

Our meter crop is 196 wide $\times$ 48 tall. That is our sampling grid.

Worked example

A digit stroke about 1.5 px wide is barely resolved. Halve the width to 98×24 and the stroke spans < 1 px — the digit becomes unreadable.

Total samples: $196 \times 48 = 9{,}408$ pixels. Halving each side: $98 \times 24 = 2{,}352$ — one quarter the samples.

- Halving *each* linear dimension quarters the pixel count (area scales as the square).
- So "a bit more resolution" costs a lot more data — budget it deliberately.

Quantization: how finely we measure brightness

- **Quantization** = rounding each sample to a finite set of levels (the bit depth).
- 8-bit = 256 levels (0–255); 1-bit = black/white; 16-bit = 65,536 (medical/RAW).
- Too few levels → visible **banding** in smooth gradients.

Note

Resolution (sampling) and bit depth (quantization) are the two axes of "how much image."

Quantization, made concrete

Worked example

An intensity of 162 stored at different bit depths, level size $= 256/2^b$:

bits b	levels 2^b	step	162 snaps to
8	256	1	162
4	16	16	160
2	4	64	192
1	2	128	255

- Fewer bits = coarser steps = bigger rounding error, then visible banding.
- 8-bit's step of 1 is below what the eye resolves on a normal display — hence the standard.

Resolution, size, and memory

- A $W \times H$ grayscale 8-bit image is $W \cdot H$ bytes; color is $3\times$ that.
- A 12-megapixel color photo $\approx$ 36 MB *uncompressed*.
- That's why formats compress — and why processing big images needs care (Week 11).

Worked example

A 4000×3000 color photo: $4000 \times 3000 = 12{,}000{,}000$ px. Grayscale: 12 MB. Color (3 bytes/px): 36 MB. Our meter crop: $9{,}408 \times 3 \approx 28$ KB.

Key idea

Know your image's shape and dtype before anything else: `img.shape`, `img.dtype`.

Color

Color: three numbers per pixel

- **RGB**: red, green, blue channels; how screens emit light.
- A color pixel is a triple (R, G, B), each 0–255.
- Grayscale collapses color to one intensity — often the first step (less data, same structure).

Math

Luminance: $Y = 0.299R + 0.587G + 0.114B$ — green dominates because our eyes do.

Luminance vs the naive average

Worked example

Take a pixel $(R, G, B) = (120, 200, 80)$.

Perceptual luminance: $Y = 0.299(120) + 0.587(200) + 0.114(80) = 35.9 + 117.4 + 9.1 = 162.4 \approx 162$.

Naive average: $(120 + 200 + 80)/3 = 400/3 = 133.3 \approx 133$.

- The two differ by ≈ 29 levels — the green-heavy pixel looks *brighter* than the flat average admits.
- Averaging under-weights green and over-weights blue relative to human vision.

Why convert to grayscale

- Many operations (edges, thresholds) need intensity, not color — 1 channel, 3× less work.
- DocScan works mostly in grayscale; color returns only when it carries information.
- But grayscale *discards* information — don't do it if hue matters (Week 8 color segmentation).

✕ Pitfall

Averaging $(R+G+B)/3$ is a common shortcut — but the weighted luminance is perceptually correct.

Other color spaces, and why

Space	Channels	Good for
RGB	red, green, blue	storage, display
Grayscale	intensity	most classical processing
HSV	hue, saturation, value	color-based selection (robust to lighting)
Lab	lightness, a, b	perceptual distance, color correction

Key idea

Choosing the right color space can make a hard problem easy — e.g. "find the red object" is trivial in HSV, painful in RGB.

HSV intuition

- **Hue**: which color (angle on the color wheel). **Saturation**: how vivid. **Value**: how bright.
- Separating color from brightness makes selection robust to shadows and lighting.
- "All the red pixels" = a hue range — one threshold, done.

Note

Hue is an angle 0–360° (OpenCV halves it to fit 0–179 in a byte). Red sits near both ends.

Value is (almost) grayscale



color meter crop (BGR)



the V (value) channel of HSV

- `V = max(R,G,B)` — the brightest channel per pixel, so it tracks intensity.
- Close to grayscale but not identical: luminance weights channels; value just takes the max.

Color helps: picking out the plate



a license-plate crop, in color



grayscale — hue is gone

- If the task were "find the colored region," HSV would isolate it by hue in one step.
- Once located, the *reading* of the characters happens in grayscale — best of both.

In-class exercise

In-class exercise (10 min)

Predict, then compute by hand

Work in your notebook first; check with code only after you have an answer.

A single pixel has $(R, G, B) = (60, 180, 90)$.

1. Compute its **luminance** $Y = 0.299R + 0.587G + 0.114B$.
2. Compute the **naive average** $(R + G + B)/3$. How far apart are they?
3. An 800×600 color image: how many bytes uncompressed? In grayscale?
4. At 2-bit depth (4 levels, step 64), what does the value 90 round to?

Debrief

- $Y = 0.299(60) + 0.587(180) + 0.114(90) = 17.9 + 105.7 + 10.3 = 133.9 \approx 134$.
- Average $= (60 + 180 + 90)/3 = 330/3 = 110$. They differ by ≈ 24 — green pulls Y up.
- $800 \times 600 = 480{,}000$ px $\Rightarrow$ 1.44 MB color, 480 KB grayscale.
- Step 64: 90 lands between 64 and 128; nearest level is 64 (since $90 - 64 = 26 < 128 - 90 = 38$).

Key idea

Every "processing" step you'll ever write is arithmetic like this, done to millions of pixels at once.

Common pitfalls this session

✗ Pitfall

Down-sizing too far destroys thin strokes — a 196×48 digit crop halved is unreadable.

✗ Wrong gray

$(R + G + B)/3$ is not perceptual gray; use the luminance weights.

✓ Good practice

Always know `shape` and `dtype` before you touch a pixel.

✓ Right tool

Pick the color space for the task: HSV to *find* color, grayscale to *read* structure.

What you should now be able to do

- Explain sampling and quantization, and the loss each one introduces.
- Relate resolution and bit depth to file size and memory with a quick calculation.
- Convert a color pixel to luminance by hand, and say why it beats the average.
- Choose between RGB, grayscale, and HSV for a given goal.

Key idea

An image is a function sampled onto a grid and rounded — two knobs, two kinds of loss.

Next session These numbers live inside a NumPy array. Session 2 loads real images in code, meets the `uint8` overflow trap, picks file formats, and traces where pixels physically come from — then sets the week's deliverable.